Coder Army

Spring Boot Annotations and Configuration

1. Why Spring Boot After Spring Core?

Till now, we learned how Spring Core manages objects using the IoC container.

In a pure Spring Core project, we usually start the container manually:

```
ApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.clas
s);
```

Then we provide a configuration class:

```
@Configuration
@ComponentScan(basePackages = "in.strikes")
public class AppConfig {
}
```

Then we fetch a bean manually and call a method:

```
OrderService orderService = context.getBean(OrderService.clas
s);
orderService.placeOrder();
```

This is useful for learning because it clearly shows how Spring works internally. But in real applications, this setup becomes repetitive.

Every Spring application needs common startup work:

```
Start application
Create Spring container
Read configuration
Scan components
Create beans
```

Coder Army

```
Inject dependencies
Load properties
Prepare application context
```

Spring Boot solves this repetitive startup problem.

> Spring Boot does not replace Spring Core.
>
> Spring Boot uses Spring Core and gives us a simpler way to start, configure, and run the Spring container.

A common beginner misunderstanding is:

```
Spring Boot = Web API
```

That is not correct.

Spring Boot can be used for:

- Console applications
- Web applications
- Database applications
- Batch applications
- Microservices

Spring Boot itself is not only about web development. Web behavior comes later when we add web-related dependencies.

---

2. Setting Up a Spring Boot Project

There are two common ways to create a Spring Boot project:

1. Create a Maven project manually and add Spring Boot dependencies.
2. Use Spring Initializr to generate a ready-made project structure.

In real projects, Spring Initializr is usually preferred because it creates the correct project structure, parent configuration, dependencies, and plugin setup.

Coder Army

3. Why Do We Use `spring-boot-starter-parent`?

In Spring Boot projects, we commonly use:

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.x.x</version>
    <relativePath/>
</parent>
```

The parent helps Maven by providing common Spring Boot project configuration.

It manages things like:

- Dependency versions
- Plugin versions
- Java version configuration
- Build defaults
- Encoding settings

The main benefit is version compatibility.

For example, if we later add dependencies like Spring JDBC, Hibernate, Validation, Jackson, or Spring Web, we do not have to manually decide every version.

Maven can ask the Spring Boot parent:

```
Which version of this dependency should I use?
```

The Spring Boot parent provides compatible versions.

This reduces version mismatch problems.

---

4. What Is a Spring Boot Starter?

A starter is a dependency shortcut.

Coder Army

Instead of adding many related dependencies one by one, we add one starter, and Maven brings the required dependencies transitively.

Example:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
</dependency>
```

`spring-boot-starter` is the basic starter for a non-web Spring Boot application.

It gives us the basic Spring Boot setup, logging, and core Spring Boot infrastructure.

Important point:

```
Starter does not mean one single library.
Starter means a collection of commonly needed dependencies.
```

---

5. Spring Core Startup vs Spring Boot Startup

In Spring Core

We manually created the Spring container:

```
ApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.clas
s);
```

We manually provided the configuration class:

```
@Configuration
@ComponentScan(basePackages = "in.strikes")
public class AppConfig {
}
```

Coder Army

We manually fetched the bean:

```
OrderService orderService = context.getBean(OrderService.clas
s);
orderService.placeOrder();
```

So in Spring Core, we usually handle these steps ourselves:

```
1. Create ApplicationContext manually
2. Provide configuration class manually
3. Fetch bean manually
4. Trigger application logic manually
```

In Spring Boot

Spring Boot gives us a standard startup mechanism:

```
SpringApplication.run(MyApplication.class, args);
```

This line starts and prepares the Spring application context for us.

---

6. Same Application in Spring Boot

A basic Spring Boot application starts like this:

```
package in.strikes;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootAppli
cation;

@SpringBootApplication
public class SpringBootCoreDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringBootCoreDemoApplication.c
```

```
lass, args);
    }
}
```

Here, we did not manually write:

```
new AnnotationConfigApplicationContext(...)
context.getBean(...)
@Configuration
@ComponentScan
```

The reason is that `@SpringBootApplication` already combines the common setup required for a Spring Boot application.

---

7. Adding Simple Components

We can create normal Spring components just like we did in Spring Core.

```
package in.strikes.service;

import org.springframework.stereotype.Component;

@Component
public class PaymentService {

    public void pay() {
        System.out.println("Payment completed");
    }
}
```

```
package in.strikes.service;

import org.springframework.stereotype.Component;

@Component
```

Coder Army

Coder Army

```
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

This is still Spring Core behavior.

Spring sees `@Component` and creates beans.

Spring sees constructor dependency and injects `PaymentService` into `OrderService`.

Spring Boot does not change dependency injection. It simply starts and configures the Spring container in a more convenient way.

---

8. How Will the Method Run?

In Spring Core, we manually fetched the bean and called the method:

```
OrderService orderService = context.getBean(OrderService.clas
s);
orderService.placeOrder();
```

In Spring Boot, we usually do not fetch beans manually from the context.

To run code after application startup, we can use `CommandLineRunner`.

```
package in.strikes.runner;

import in.strikes.service.OrderService;
```

Coder Army

```
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class AppRunner implements CommandLineRunner {

    private final OrderService orderService;

    public AppRunner(OrderService orderService) {
        this.orderService = orderService;
    }

    @Override
    public void run(String... args) {
        orderService.placeOrder();
    }
}
```

`CommandLineRunner` is a Spring Boot interface.

Its `run()` method executes after the Spring application context is created and ready.

This is useful for console-based Spring Boot applications, testing startup logic, or running some code immediately after the application starts.

---

9. Understanding `SpringApplication.run()`

The most important line in a Spring Boot application is:

```
SpringApplication.run(SpringBootCoreDemoApplication.class, ar
gs);
```

This line:

- Starts the Spring Boot application
- Creates the application context

Coder Army

- Reads configuration and properties
- Performs component scanning
- Creates beans
- Applies auto-configuration
- Injects dependencies
- Runs startup hooks like `CommandLineRunner`

`run()` also returns the application context:

```
ConfigurableApplicationContext context =
        SpringApplication.run(SpringBootCoreDemoApplication.c
lass, args);
```

Technically, we can still fetch beans manually:

```
OrderService orderService = context.getBean(OrderService.clas
s);
orderService.placeOrder();
```

But in real Spring Boot applications, this is not the usual approach.

The preferred approach is:

```
Let Spring inject dependencies wherever they are needed.
```

---

10. Understanding `@SpringBootApplication`

`@SpringBootApplication` is not a small annotation. It is a combination annotation.

It roughly combines:

```
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan
```

Coder Army

So this:

```
@SpringBootApplication
public class SpringBootCoreDemoApplication {
}
```

is conceptually similar to:

```
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan
public class SpringBootCoreDemoApplication {
}
```

Meaning:

Annotation	Purpose
`@SpringBootConfiguration`	Marks the main configuration class of the Spring Boot application
`@EnableAutoConfiguration`	Enables Spring Boot's automatic configuration mechanism
`@ComponentScan`	Scans the current package and subpackages for Spring components

We already studied `@Configuration` and `@ComponentScan` in Spring Core.

The major new concept here is:

```
@EnableAutoConfiguration
```

---

11. `@SpringBootConfiguration`

`@SpringBootConfiguration` is Spring Boot's version of a configuration marker.

In Spring Core, we used:

```
@Configuration
public class AppConfig {
```

Coder Army

```
    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

This class tells Spring:

```
This class contains bean definitions.
```

In Spring Boot, the main class indirectly gets `@SpringBootConfiguration` because of `@SpringBootApplication`.

That means the main class can also define beans:

```
@SpringBootApplication
public class SpringBootCoreDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringBootCoreDemoApplication.c
lass, args);
    }

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

This works because the main class is also treated as a configuration class.

However, in real projects, we usually keep the main class clean and create separate configuration classes when needed.

Example:

Coder Army

```java
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

Simple difference:

```
@Configuration = Regular Spring configuration class
@SpringBootConfiguration = Main Spring Boot application configuration marker
```

`@SpringBootConfiguration` also helps Spring Boot tests locate the main application configuration automatically.

---

12. `@ComponentScan` in Spring Boot

In Spring Core, we manually wrote:

```java
@Configuration
@ComponentScan(basePackages = "in.strikes")
public class AppConfig {
}
```

This told Spring:

```
Go inside the in.strikes package and find classes marked with @Component, @Service, @Repo
sitory, or @Controller.
```

In Spring Boot, we usually do not write `@ComponentScan` manually because it is already included inside `@SpringBootApplication`.

Coder Army

By default, Spring Boot scans the package where the main application class is present and all its subpackages.

Example package structure:

```
in.strikes
├── SpringBootCoreDemoApplication.java
├── service
│   ├── PaymentService.java
│   └── OrderService.java
└── runner
    └── AppRunner.java
```

Here, the main class is inside:

```
in.strikes
```

So Spring Boot scans:

```
in.strikes and all packages under it
```

This is why Spring Boot applications usually keep the main application class in the root package.

Correct structure:

```
in.strikes
├── SpringBootCoreDemoApplication.java
├── controller
├── service
├── repository
└── config
```

Less ideal structure:

```
in.strikes.app
└── SpringBootCoreDemoApplication.java
```

Coder Army

```
in.strikes.service
└── PaymentService.java
```

In the second structure, `PaymentService` may not be scanned automatically because it is outside the main class package hierarchy.

---

13. Can We Manually Change Component Scanning?

Yes.

If required, we can manually provide base packages:

```java
@SpringBootApplication(scanBasePackages = "in.strikes")
public class SpringBootCoreDemoApplication {
}
```

This tells Spring Boot to scan from `in.strikes` even if the main class is placed somewhere else.

However, the better practice is to place the main class in the root package and let Spring Boot scan naturally.

---

14. `@EnableAutoConfiguration`

`@EnableAutoConfiguration` tells Spring Boot:

```
Look at my project setup and automatically configure useful things for me.
```

Spring Boot checks:

- Which dependencies are present
- Which classes are available on the classpath
- Which beans already exist
- Which properties are configured

Then it creates useful default beans only when needed.

Coder Army

For example, Spring Boot may check:

```
Is a web dependency present?
Is a database dependency present?
Is Spring Security present?
Has the developer already created a custom bean?
```

Based on these checks, Spring Boot applies its pre-written configuration classes.

Examples of common auto-configuration classes:

```
DataSourceAutoConfiguration
WebMvcAutoConfiguration
JacksonAutoConfiguration
TaskExecutionAutoConfiguration
```

Simple meaning:

```
@EnableAutoConfiguration = Apply Spring Boot's ready-made configurations when conditions
match.
```

---

15. How Auto-Configuration Works Internally

When the application starts:

```
SpringApplication.run(SpringBootCoreDemoApplication.class, ar
gs);
```

Spring Boot sees:

```
@SpringBootApplication
```

Inside it, auto-configuration is enabled:

```
@EnableAutoConfiguration
```

CoderArmy

Then Spring Boot considers auto-configuration classes.

These classes are not blindly applied. They are applied only when their conditions match.

Conceptually, an auto-configuration class looks like this:

```
@AutoConfiguration
@ConditionalOnClass(SomeLibrary.class)
public class SomeLibraryAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public SomeService someService() {
        return new SomeService();
    }
}
```

Meaning:

```
If SomeLibrary is present in the project
AND the developer has not already created SomeService bean
THEN create a default SomeService bean.
```

This is the core idea of Spring Boot auto-configuration.

---

16. Manual Configuration vs Auto-Configuration

Manual Configuration

In Spring Core, the developer writes the configuration manually:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
```

Coder Army

```
        return new PaymentService();
    }
}
```

Here, the developer directly tells Spring:

```
Create this bean.
```

Auto-Configuration

In Spring Boot, many configuration classes are already written for us.

They are provided by Spring Boot or sometimes by third-party starters.

Spring Boot applies them only when the right conditions are satisfied.

So auto-configuration does not mean random configuration.

It means:

```
Pre-written configuration + condition-based activation
```

---

17. Classpath-Based Decision Making

Classpath means the set of classes and libraries available to the application at runtime.

In Maven terms:

```
Dependency added in pom.xml
        ↓
Maven downloads JAR files
        ↓
Those JARs become available to the application
        ↓
Spring Boot checks the available classes
        ↓
Auto-configuration decisions are made
```

When we add a dependency, we indirectly give Spring Boot a signal.

Coder Army

Example:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
</dependency>
```

This tells Spring Boot:

```
This is a basic Spring Boot application.
```

But if we do not add:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

then Spring Boot does not treat the application as a web application.

It checks:

```
Do I see web server classes? No.
Do I see Spring MVC web classes? No.
Should I start Tomcat? No.
```

So the application starts like a normal console application.

You will not see:

```
Tomcat started on port 8080
```

This is because no web-related classes are present on the classpath.

---

18. What Happens When We Add Web Dependency?

Coder Army

Later, when we add:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Maven brings web-related libraries.

Then Spring Boot sees:

```
Web classes are present.
Embedded server support is present.
Spring MVC infrastructure is present.
```

Now Spring Boot understands:

```
This looks like a web application.
```

So it configures web-related beans and starts an embedded server such as Tomcat.

That is why a Spring Boot web application keeps running and listens for HTTP requests.

---

19. Three Main Checks in Auto-Configuration

Spring Boot commonly checks three things:

```
1. Is the required class present?
2. Is the required bean missing?
3. Is the required property enabled?
```

Conceptual example:

```
@AutoConfiguration
@ConditionalOnClass(PaymentGateway.class)
```

Coder Army

```java
public class PaymentAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public PaymentGateway paymentGateway() {
        return new DefaultPaymentGateway();
    }
}
```

Meaning:

```
If PaymentGateway class exists
AND the developer has not created a PaymentGateway bean
THEN create a default PaymentGateway bean.
```

This is why Spring Boot feels automatic but still remains controlled.

---

20. `@ConditionalOnClass`

`@ConditionalOnClass` is a condition that matches only when a particular class is present on the classpath.

Simple meaning:

```
Apply this configuration only if a specific class or library is available in the project.
```

Example idea:

```java
@ConditionalOnClass(SomeLibrary.class)
```

This means:

```
Only activate this configuration if SomeLibrary is present.
```

This is how Spring Boot detects what kind of application we are building.

CoderArmy

21. `@ConditionalOnMissingBean`

`@ConditionalOnMissingBean` is a condition that matches only when a required bean is not already present in the Spring container.

Simple meaning:

```
Create this bean only if the developer has not already created one.
```

This is very important because Spring Boot does not want to override our custom configuration unnecessarily.

Example:

```
@Bean
@ConditionalOnMissingBean
public SomeService someService() {
    return new DefaultSomeService();
}
```

Meaning:

```
If the developer has not created SomeService bean, then create the default one.
```

This is why our own beans usually get preference over Boot's default beans.

---

22. Our Beans vs Auto-Configured Beans

In a Spring Boot application, beans can come from two main sources.

1. Beans Created from Our Code

These are developer-defined beans.

Example using `@Component`:

```
@Component
public class PaymentService {
```

Coder Army

```
    public void pay() {
        System.out.println("Payment done");
    }
}
```

Example using `@Bean`:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

These beans are usually discovered through component scanning or explicit configuration.

2. Beans Created by Spring Boot Auto-Configuration

These beans come from Spring Boot's pre-written configuration classes.

Conceptual example:

```
@AutoConfiguration
@ConditionalOnClass(SomeLibrary.class)
public class SomeLibraryAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public SomeService someService() {
        return new DefaultSomeService();
    }
}
```

Coder Army

These are not classes we write in our application.

They are provided by Spring Boot or by a third-party starter.

---

23. Complete Spring Boot Startup Flow

A simplified Spring Boot startup flow looks like this:

```
Application starts
        ↓
SpringApplication.run() is called
        ↓
Application context is created
        ↓
User configuration is processed
        ↓
Component scanning finds our classes
        ↓
User-defined beans are registered
        ↓
Auto-configuration classes are considered
        ↓
Conditions are checked
        ↓
Missing default beans are created
        ↓
Dependencies are injected
        ↓
Application context becomes ready
        ↓
CommandLineRunner/ApplicationRunner runs, if present
```

---

24. Important Difference: `@ComponentScan` vs `@EnableAutoConfiguration`

These two annotations solve different problems.

Annotation	What It Does
`@ComponentScan`	Finds our application classes marked with `@Component`, `@Service`, `@Repository`, `@Controller`, etc.
`@EnableAutoConfiguration`	Applies Spring Boot's ready-made configurations based on classpath, beans, and properties.

Coder Army

Simple comparison:

```
@ComponentScan finds our code.
@EnableAutoConfiguration applies Boot's default setup.
```

Both are included inside `@SpringBootApplication`.

---

25. Final Takeaways

- Spring Boot does not replace Spring Core.
- Spring Boot uses Spring Core internally.
- Spring Core gives us IoC and dependency injection.
- Spring Boot gives us a standard way to start and configure the application.
- `SpringApplication.run()` creates and prepares the application context.
- `@SpringBootApplication` combines `@SpringBootConfiguration`, `@EnableAutoConfiguration`, and `@ComponentScan`.
- `@ComponentScan` finds our classes.
- `@EnableAutoConfiguration` applies Spring Boot's ready-made configurations.
- Auto-configuration works through conditions.
- `@ConditionalOnClass` checks whether a class or library is present.
- `@ConditionalOnMissingBean` creates a bean only when the developer has not already created one.
- Adding dependencies changes the classpath, and Spring Boot uses that classpath to decide what to configure.
- A basic Spring Boot starter does not start a web server.
- A web starter adds web-related classes, so Spring Boot starts an embedded server.

---

26. One-Line Summary

Spring Boot is not magic. It is Spring Core plus a smart startup and configuration system that uses dependencies, properties, and conditions to prepare the application automatically.

Coder Army